Birds Image Classification Kaggle Competition

Kristof BALAZS
CentraleSupélec
kristof.balazs@student-cs.fr

Stefanos KYPRITIDIS
CentraleSupélec
stefanos.kypritidis@student-cs.fr

Abstract

We study fine-grained bird species recognition on a 20-class CUB subset, where subtle part-level cues must be extracted under large pose/scale/background variation from only 1082 training and 103 validation images; because the validation split is small and exhibits class-proportion drift, we select models primarily via stratified 5-fold cross-validation. We benchmark classical descriptors against transfer learning with pretrained `timm` *CNN/Transformer backbones used as frozen feature extractors with lightweight heads (linear, ArcFace), and we mitigate background shortcutting with bird-centered cropping, aspect-ratio preserving padding, and higher input resolutions (384–448). Using FiftyOne for qualitative and class-wise error analysis, we find that residual errors concentrate on a few near-identical species pairs (notably American vs. Fish Crow), motivating targeted add-ons (pseudo-labeling, a binary crow separator). Pretrained representations dominate classical baselines, and the most reliable gains come from preprocessing and resolution; however, we observe a generalization trade-off: EVA-02 + ArcFace reaches 93.0% on public validation but drops to 85.5% on private test, while a simpler ViT-384 with a linear head remains stable at 90.0% on both.*

1. Introduction

Fine-grained bird species recognition is challenging because classes differ by subtle, localized cues (plumage micro-textures, beak curvature) under large nuisance variation in pose, scale, illumination, and background. In this project, we build an image classifier for a 20-class subset of CUB images, evaluated with top-1 accuracy on a hidden test set. A key constraint is that species supervision must come only from the provided labeled dataset, which makes transfer learning and careful use of the available labels central to performance.

We first perform an exploratory analysis of the provided metadata and images to guide evaluation and preprocessing. The labeled data consists of 1082 training images and a small validation split of 103 images. While the training set is close to balanced (51–58 images per class), the validation split exhibits noticeable class proportion drift (2–8 images per class), making single-split validation accuracy high-variance; therefore, we rely primarily on stratified K-fold cross-validation on the training set for model selection, using the validation split mainly for sanity checks. Images are pre-resized with a maximum side of 500 pixels and show strong aspect-ratio variability (median train size 500×375, aspect ratios in $[0.38, 2.17]$), motivating aspect-ratio preserving resizing with padding and bird-centered cropping to reduce background shortcutting.



Figure 1. Variety of bird species

Guided by these findings, we benchmark approaches ranging from classical descriptors to modern pretrained backbones, and complement aggregate metrics with class-wise and qualitative error analysis (FiftyOne) to reveal systematic confusions between visually similar species. This analysis motivates targeted improvements beyond a single global model, including pseudo-labeling and a specialized crow separator for the most persistent failure modes. The full implementation is publicly available on our GitHub repository: https://github.com/stef4k/image-classification-birds

2. Error Analysis and Qualitative Inspection

To go beyond aggregate metrics (accuracy, macro-F1), we used the *FiftyOne* App for interactive inspection of validation predictions. For each validation image, we

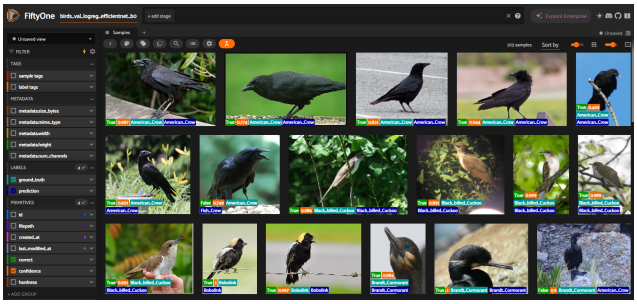


Figure 2. FiftyOne App used for qualitative error analysis on the validation split.

store the `ground_truth` label, the model `prediction`, a boolean `correct` flag, and a `confidence` score when available (e.g., for Logistic Regression using `predict_proba`). This enables rapid identification of failure cases and helps explain *why* the model confuses specific bird species.

Figure 2 shows the FiftyOne grid view, where each sample displays (i) a correctness flag, (ii) he corresponding confidence score, (iii) the ground-truth label, (iv) the predicted label. In our visualization, the small **green** sticker (`True`/`False`) indicates whether the prediction matches the ground truth (`correct`); the **orange** sticker shows the model confidence; the **light sea blue** text label corresponds to the ground-truth class; and the **dark blue** text label corresponds to the predicted class. A key advantage of FiftyOne is that all stored attributes are immediately filterable from the left panel. For example, we can filter only incorrect predictions (`correct = false`), restrict the view to a particular species, or sort by confidence (or hardness) to surface the most uncertain and ambiguous cases. This makes FiftyOne a practical tool for understanding model behavior, diagnosing systematic confusions between visually similar classes, and verifying whether errors are driven by background, viewpoint, or fine-grained appearance differences.

3. Experiments

3.1. Pretrained Embeddings with Linear Classifiers

In addition to fully hand-crafted baselines, we evaluated a lightweight yet strong approach based on fixed deep feature embeddings extracted from pretrained convolutional backbones, followed by a simple linear classifier. Concretely, each image is passed through a frozen backbone network (EfficientNet-B0 or ResNet-50), and the resulting embedding vector is used as input to a linear model (Linear SVM or Logistic Regression). This design isolates the contribution of representation learning (pretrained backbone) from classification capacity (linear decision boundary), while remaining computationally efficient and easy to reproduce.

Embedding extraction and caching. Embeddings are computed once for a given configuration (backbone, cropping option, and preprocessing/augmentation settings) and cached to disk to avoid repeated forward passes through the backbone. The preprocessing pipeline is configurable and can include an augmentation mode (random resized crop, horizontal flip, and rotation) for embedding extraction on the training set; in all cases, the evaluation stage uses a deterministic transform for consistency. This caching mechanism enables rapid experimentation over a large hyperparameter space without repeatedly recomputing features.

Training and evaluation protocol. For model selection, we use 5-fold stratified cross-validation on the training embeddings. Each fold trains a linear classifier on the training split and evaluates on the held-out fold, reporting accuracy and macro-F1. After cross-validation, a final model is trained on the full training set and saved together with a run-specific metadata file that records the full configuration (classifier type, regularization strength C, backbone choice, cropping flag, preprocessing parameters, and cross-validation metrics). Validation performance on the held-out validation set is computed using the corresponding evaluation script, which reloads the saved model and reproduces the exact embedding transform configuration from the metadata.

Hyperparameter sweep with Weights & Biases. To systematically identify strong configurations, we implemented a grid sweep over the main hyperparameters (e.g., classifier type and regularization strength C, solver and penalty options for Logistic Regression, loss and dual settings for Linear SVM, as well as backbone choice and preprocessing options). All runs are logged to Weights & Biases (W&B), enabling large-scale comparison across configurations and visualization of cross-validation performance through interactive dashboards.

The full Grid-Search sweep for pretrained embeddings combined with a Linear SVM is available in the W&B dashboard[1] (the code implementation can be found in the Github branch `sweep_dev`), where the best-performing configuration reaches a mean cross-validation accuracy of 0.888, as highlighted in the scatterplot view. Similarly, the Grid-Search sweep for pretrained embeddings with Logistic Regression can be inspected in a separate W&B dashboard[2], with the strongest configuration achieving a mean cross-validation accuracy of 0.8938. These dashboards provide a transparent overview of the explored hyperparameter space

[1] https://api.wandb.ai/links/stef4k-centralesupelec/m3yq79nt

[2] https://api.wandb.ai/links/stef4k-centralesupelec/c38ri26i

and clearly illustrate the relative performance of two powerful linear classifiers on fixed deep representations.

3.2. Backbone Selection and Architecture

To maximize our choices and control over the exact backbones on fine-grained classification with limited training data, we used the `timm` (PyTorch Image Models) library to access State-of-the-Art (SOTA) pretrained backbones [11]. This made it easy for us to experiment with a variety of approaches (and also different versions for each) that we read about. We focused on Transfer Learning, using models pretrained on large-scale datasets (ImageNet-1k and ImageNet-21k). We experimented with a variety of architectures, ranging from efficient CNNs to large Vision Transformers (ViTs).

To properly evaluate the gain from more advanced models, we progressively increased model complexity and input resolution, because as mentioned before, this fine-grained recognition benefits significantly from higher pixel density. The backbones were initialized with `num_classes=0` to be pure feature extractors, producing a fixed-size embedding vector.

3.2.1. The Rationale for Frozen Backbones

Given the limited size of the training dataset (20 classes, 51-58 images in each, 1083 in total) relative to the complexity of the task (distinguishing visually similar bird species), training deep architectures from scratch would lead to severe overfitting. Furthermore, fine-tuning the entire parameter space of large models (e.g., ViT-Large or ConvNeXt-Large) requires substantial VRAM and computational time and it did not improve the results based on our experiments.

Therefore, we used a **frozen backbone** strategy. By freezing the parameters of the feature extractor, we treat the backbone as a fixed non-linear function that projects input images into a high-dimensional semantic latent space. Only the classification head (discussed in later sections) is trainable. This approach has two main advantages for our task:

1. **Regularization:** It keeps the general-purpose feature detectors learned on ImageNet (such as edge detectors, texture filters, and part detectors) and it prevents them from being destroyed by gradients calculated on a small, specific dataset.
2. **Efficiency:** It eliminates the need to store gradients for the backbone during backpropagation, which reduces memory footprint and allows for larger batch sizes and higher resolution inputs (e.g., 384×384 or 448×448), which are important for detecting fine-grained details like feather patterns.

3.2.2. Backbone Architectures

We evaluated backbones of increasing complexity, the specific models selected for experimentation are detailed below.

ResNet-50 and EfficientNet-B0 (The Baselines). We used ResNet-50 [5] as our main convolutional baseline. It uses residual skip connections to allow for deep gradient flow. Although it is robust, standard ResNets often do not have the capacity to capture long-range dependencies compared to modern architectures. We also tired EfficientNet-B0 [10], which uses compound scaling to balance depth, width, and resolution. It is a lightweight baseline that is very efficient computationally.

ConvNeXt (Modernizing the CNN). To bridge the gap between pure CNNs and Transformers, we use **ConvNeXt** [6]. This architecture keeps the standard convolutional inductive bias (translation invariance) but uses a "Transformer-style" design: it replaces standard convolutions with depthwise convolutions, uses larger kernels (7×7) to make the receptive field larger, and changes BatchNorm to LayerNorm. We used the `convnext_base` variant pre-trained on ImageNet-21k [7], which has a richer feature representation than standard 1k pre-training (really important given our task as they have more bird classes and a larger number of images in each).

ConvNeXt V2 (Self-Supervised Learning). We also experimented with **ConvNeXt V2** [12], specifically the Base and Large variants (`convnextv2_base_384`, `convnextv2_large_384`). In V2, they add a Global Response Normalization (GRN) layer so the model does not rely too much on a few channels and uses them more evenly. More importantly, these weights are pre-trained using a fully convolutional masked autoencoder (FCMAE) framework. This self-supervised pretraining helps the network learn whole-object representations from incomplete views, which is useful for bird classification since birds are often partly hidden by leaves or only parts of them are visible on the picture (as in the case of many images in our dataset).

CAFormer (The Hybrid Approach). From the literature, we realized that CNNs are good at capturing local details such as textures and feathers, while Transformers are better at modeling global information like shape and pose. Based on this, we tried **CAFormer-B36**. CAFormer is a "MetaFormer" architecture that used Convolutional blocks in the early stages to extract low-level features, and Self-Attention blocks in the deeper stages to capture global semantic relationships [13]. This hybrid approach makes it theoretically ideal for fine-grained classification at higher resolutions (384×384).

Vision Transformers (ViT). Finally, we experimented with a purely attention-based model. We selected

ViT-SO150M (Source-Only, 150M parameters), specifically `vit_so150m2_patch16_reg1_gap_384`. Unlike CNNs, Vision Transformers split an image into fixed-size patches (e.g., 16×16 pixels) and treat them as a sequence processed with multi-head self-attention [3]. This allows the model to relate information from different (and distant) regions of the image, such as the correlation of beak shape with tail patterns.

EVA-02 (State-of-the-Art). Our most advanced model was **EVA-02 Large** (`eva02_large_448`). EVA-02 [4] is a Transformer-based model pre-trained to reconstruct features from a vision-language model (CLIP) using masked image modeling. This pretraining helps the model learn meaningful visual features. We used an input resolution of 448×448, the highest in our experiments, to better capture small visual differences between closely related bird species that are often misclassified (e.g., Fish Crow and American Crow or Brewer Blackbird and Rusty Blackbird in our case).

The TIMM model cards, along with a summary of the backbones used in our experiments are detailed in Table 1. The exact implementation is in the Python script *embedder.py*

3.3. Head Experiments

While the backbone extracts high-level features from the images, the classification head maps these features to the 20 bird species. We tried three different head designs, ranging from standard non-linear classifiers to more structured metric learning approaches.

3.3.1. Multi-Layer Perceptron (MLP)

Our initial approach used a custom MLP head, designed to introduce non-linearity before the final classification. The architecture had a projection layer, Batch Normalization, GELU activation, and Dropout.

$$\mathbf{y} = \text{Linear}(\text{Dropout}(\text{GELU}(\text{BatchNorm}(\text{Linear}(\mathbf{x})))))) \tag{1}$$

We thought that the frozen backbone features might not be perfectly linearly separable and needed non-linear processing to distinguish subtle species differences. We used GELU activations to match the internal activations of modern backbones like ConvNeXt.

In practice, this head performed quite poorly. Adding trainable layers on top of a frozen backbone made optimization harder. Batch Normalization was also unstable because of the small batch sizes required by high-resolution images. Moreover, the added non-linearity seemed to disrupt the semantic structure learned by the pre-trained backbone. The final result was always slower convergence and lower final accuracy compared to simpler heads.

3.3.2. Simple Linear Head

We reverted to the standard approach used for classification with pre-trained backbones: a single Linear Layer.

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{2}$$

State-of-the-art backbones like EVA-02 and ConvNeXt V2 are pre-trained on massive datasets (ImageNet-21k, M-38M) to output latent spaces that are already highly linearly separable. A simple linear transformation keeps the geometric relationships between feature vectors.

This head was a reliable baseline, with good performance in almost every experiment. It is simple, efficient, and fast to train, which made it convenient for testing different data augmentations and image sizes.

3.3.3. ArcFace (Additive Angular Margin)

To improve performance beyond what linear classifiers could achieve, we implemented an **ArcFace** head [2]. Unlike standard cross-entropy loss, which separates classes based on Euclidean distance, ArcFace projects features onto a hypersphere and adds an explicit angular margin between classes.

The core idea is to replace the standard dot product with **cosine similarity** by normalizing both the feature vectors and the class weights:

$$\cos(\theta_j) = \frac{\mathbf{W}_j^T \mathbf{x}_i}{\|\mathbf{W}_j\| \|\mathbf{x}_i\|} \tag{3}$$

During training, an additive angular margin m is added to the angle θ_{y_i} of the ground truth class before computing the cosine similarity. The modified loss function is defined as:

$$L_{\text{Arc}} = -\log \frac{e^{s\cos(\theta_{y_i}+m)}}{e^{s\cos(\theta_{y_i}+m)} + \sum_{j \neq y_i} e^{s\cos\theta_j}} \tag{4}$$

where s is a scaling factor (fixed at 30.0) and m is the margin (fixed at 0.50).

Fine-grained bird classification involves high **inter-class similarity** (e.g., distinguishing a Fish Crow from an American Crow). With a standard linear head, feature clusters can overlap near the decision boundary. ArcFace forces the model to compress features of the same class into tighter clusters while increasing the separation between different classes on the hypersphere. This results in clearer decision boundaries.

We found that ArcFace improves performance mainly when used with high-resolution inputs (384×384 or 448×448). At lower resolutions (224×224), the extracted features are coarser, which makes angular margin–based learning more difficult. On the contrary, higher-resolution inputs

Table 1. Comparison of backbones used in our experiments. Top-1 accuracy and parameter counts are based on the `timm` benchmark on the ImageNet-1k validation set. Pre-training datasets include IN-1k (1.3M images), IN-12k (12M images), IN-22k (14M images), and M-38M (merged 38M dataset). By clicking on the model name, you can navigate to the TIMM card of the model.

Model Name	Resolution	Params (M)	Top-1 Acc (%)	Pretrain Data	Architecture	Key Feature
`efficientnet_b0`	224^2	5.3	77.70	IN-1k	CNN	Compound scaling baseline
`resnet50`	224^2	25.6	81.22	IN-1k	CNN	Residual connections
`convnext_base`	224^2	88.6	86.30	IN-22k	CNN	Modernized CNN design
`convnextv2_base`	384^2	88.7	87.64	IN-22k	CNN	Fully conv. MAE + FCMAE
`vit_so150m2`	384^2	136.3	87.93	IN-12k	ViT	Regularized ViT
`caformer_b36`	384^2	98.8	88.06	IN-22k	Hybrid	Conv + attention MetaFormer
`convnextv2_large`	384^2	198.0	88.18	IN-22k	CNN	Fully conv. MAE + FCMAE
`eva02_large`	448^2	305.1	90.06	M-38M	ViT	MIM with CLIP targets (SOTA)

keep finer visual details, which results in more discriminative feature representations. In higher resolutions, ArcFace is better able to separate closely related species.

3.4. Image Resizing, Augmentation, and other Experiments

3.4.1. Image Cropping and Resizing

Our experiments showed that removing background noise was very important for model performance. Feeding the entire image often led the model to overfit on background context instead of the actual specific bird features. To tackle this problem, we implemented a Hybrid Cropping Pipeline.

The pipeline operates in three stages:

1. **Ground Truth (GT) for Test and Validation Images:** For the training and validation sets, we used the official CUB metadata (`bounding_boxes.txt`).
2. **Faster R-CNN for Test Images:** For the test set (where GT is unavailable), we used a pre-trained `Faster R-CNN (ResNet50-FPN)` detector. We configured the detector to specifically target COCO Class ID 16 (Bird) with a confidence threshold of 0.6. To keep some of the context, we added a dynamic 5% padding to all bounding boxes. The implementation is in the script *crop_with_pretrained.py*.
3. **Manual Corrections:** Post-processing, we looked at each image of the cropped test set. We found around 20 cases where the AI detector was slightly wrong (e.g., cropping a branch instead of the bird or missing the bird entirely). These images were manually re-cropped.

The goal of this strategy is to make sure that our models focus on fine-grained features (beak shape, plumage patterns) instead of environmental features.

To prepare the non-uniform cropped images for the neural network input (it requires square tensors), we use a custom `SquarePad` transformation. As standard resizing squashes or stretches images to fit dimensions (e.g., 224×224), our approach keeps the original aspect ratio to prevent distortion of features like beak shape and leg length. The image is rescaled so that its longest edge matches the target resolution (e.g., 448 pixels) using Bicubic interpolation. The shorter dimension is padded with a neutral gray value (128) to square the image.

3.4.2. Data Augmentation

We are using a quite conservative augmentation strategy that focused on preserving geometric structure rather than introducing strong augmentations. The final pipeline includes `SquarePad` (letterbox resizing), `RandomHorizontalFlip`, and mild `RandomRotation` ($\pm 15°$).

We chose augmentations that maintain the shape and color, which are important for fine-grained bird classification. We tried, but discarded color augmentations, as color in a variety of body parts is an important cue for many species.

3.4.3. Optimizer and Weight Decay

We used the **AdamW** optimizer with an initial learning rate of 1e-3 and a weight decay of 1e-4. AdamW was selected over standard SGD and Adam for its decoupled weight decay implementation, which has better regularization on the fine-grained CUB dataset. To stabilize training, we used a cosine annealing learning rate scheduler, which gradually reduces the learning rate to zero. This helps the model converge more smoothly and lowers the risk of overfitting.

3.4.4. Loss and Label Smoothing

Due to the high inter-class similarity in our task, models can become overly confident in their predictions, which may lead to overfitting and just poor generalization. To mitigate this, we used label smoothing with a factor of $\alpha = 0.1$. Instead of assigning a probability of 1.0 to the correct class, label smoothing assigns a target probability of 0.9 to the ground-truth class and distributes the remaining probability uniformly across the other classes.

3.5. Other Experiments (Failed)

This subsection presents several approaches that were initially well motivated but did not lead to performance improvements in practice. A key reason is the limited size of

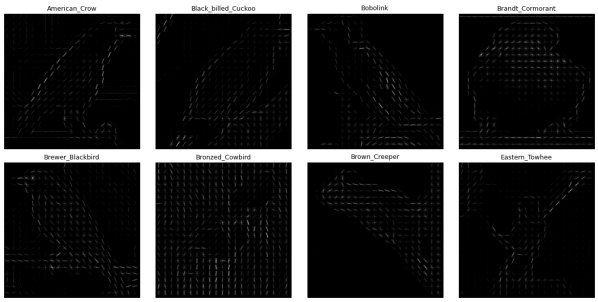


Figure 3. Visualization of HOG descriptors for several bird images. Brighter regions indicate stronger gradient responses, highlighting edges and coarse shape structure while discarding color information.

the test set, which makes accuracy an unreliable metric for drawing conclusions. We think these are very promising methods for lagrer test sets.

3.5.1. HOG & Logistic Regression

As a fully hand-crafted and interpretable baseline, we evaluated *Histogram of Oriented Gradients* (HOG) features with multinomial Logistic Regression. HOG encodes local edge orientation statistics and is effective for capturing coarse shape cues [1]. We extract HOG using `skimage.feature.hog` with standard block normalization on grayscale images (shape/edge information only). To optionally add coarse color information, we concatenate a global HSV histogram; this adds overall color distribution but no spatial layout. Features are standardized and classified with class-balanced Logistic Regression.

All images are resized to a fixed resolution. Hyperparameters (HOG settings and the Logistic Regression regularization) were selected via a grid-search using 5-fold stratified cross-validation on the training set. The implementation is provided in *train_hog_logreg.py* and *hog_logreg_training.ipynb*.

Figure 3 visualizes typical HOG responses: gradients highlight silhouettes and strong internal edges, while discarding fine texture and color patterns. Quantitatively, the method achieved **22%** mean cross-validation accuracy on training and **20%** on validation. Extensive tuning did not meaningfully improve results, indicating that edge/shape statistics (even with a global color histogram) are fundamentally insufficient for fine-grained bird species recognition, where discriminative cues are dominated by localized plumage patterns, micro-textures, and subtle part-level morphology rather than global shape. Despite low performance, this baseline is useful as a transparent lower bound and as evidence of the representational gap between classical descriptors and modern pretrained features in fine-grained settings.

3.5.2. Fisher Vector Encoding with Linear SVM

To build a stronger classical baseline than HOG, we implemented a Fisher Vector (FV) pipeline. FV represents an image by aggregating local descriptors (dense SIFT) through deviations from a generative model (a diagonal-covariance GMM), yielding a high-dimensional orderless encoding that has historically been competitive with linear classifiers [8]. We follow the standard "improved FV" recipe (power/signed square-root normalization followed by ℓ_2 normalization) using the `scikit-image` Fisher Vector implementation and its GMM routine [9].

Images are resized to a fixed resolution, and SIFT descriptors are extracted densely over a regular grid. We optionally apply RootSIFT normalization (L1 followed by elementwise square-root). A K-component GMM is fitted on a random subset of descriptors sampled from training images. For each image, we compute the Fisher Vector encoding (first- and second-order statistics w.r.t. the GMM) and train a class-balanced linear SVM on the resulting features. Hyperparameters were chosen empirically and performance was reported via 5-fold stratified cross-validation on the training set, followed by evaluation on the held-out validation split.

This approach achieved **38.4%** cross-validation accuracy and **35.9%** validation accuracy, substantially below modern pretrained feature extractors. To inspect separability qualitatively, we projected $N = 1000$ Fisher Vectors with t-SNE (perplexity $= 30$, PCA init). Figure 4 shows strong class mixing with no clear species-level cluster structure, consistent with the limited classification performance. Overall, the failure aligns with the demands of fine-grained recognition: hand-crafted local descriptors and orderless aggregation capture generic texture statistics but lack reliable part alignment and sensitivity to subtle, localized markings under pose/lighting variation, which are critical for separating closely related bird species.

3.5.3. Hierarchical "Mixture of Experts"

To reduce confusion between visually similar species, such as American vs. Fish Crows and Rusty vs. Brewer's Blackbirds, we tried out a hierarchical hard-routing system.

Our Vision Transformers (like EVA-02) are very good at capturing global shape and context but they often smooth over high-frequency local details. CNNs have an inductive bias toward local edge and texture detection, which makes them better at distinguishing subtle features such as beak curvature or plumage texture.

We trained two specialized ConvNeXt V2 Large models just on the binary classification tasks for Crows and Blackbirds. During inference, if the General Model predicted outside of these classes, its output was accepted. If it predicted a Crow or Blackbird, the image was rerouted to the respective Expert CNN for the decision.



Figure 4. t-SNE visualization of a subset of Fisher Vectors ($N = 1000$). Points are colored by class label (20 classes).

Unfortunately, this approach did not outperform the single-model baseline. This result suggests to us that performance is limited by the data rather than the model architecture. The expert models were trained on the same class-imbalanced subsets as the general model, and therefore did not learn additional discriminative features. Distinguishing American Crows for example is a difficult task even for experts (identifying them based on sound is the easiest), and our training data is too limited to achieve this.

3.5.4. Ensemble of Our Best Models

We also attempted to combine the strengths of our top-performing architectures into a single predictive system using a Soft Voting Ensemble. Our thought was that different architectures (Transformers vs. CNNs) would make uncorrelated errors, which would allow them to correct each other on difficult samples.

We aggregated the softmax probability distributions from three different models:

1. EVA-02 Large (ArcFace): Our strongest individual model, trained at 448×448 resolution.
2. ConvNeXt V2 Large (Linear): A pure CNN architecture trained at 384×384.
3. ViT-SO400M (Linear): A standard Vision Transformer trained at 384×384.

The limited improvement was mainly due to calibration mismatch between the models. The EVA-02 model trained with ArcFace produced quite confident predictions, often with probabilities over 0.99, but the linear models were less confident. When using simple probability averaging, this imbalance caused the EVA-02 model to dominate the ensemble.

Despite this, the ensemble was still a useful safety net for us: the CNN (ConvNeXt) helped stabilize predictions on texture-heavy samples where the transformer models were less certain.

3.5.5. Partial Unfreezing of Backbones

To avoid the compute and overfitting risks of full fine-tuning, we tried surgical fine-tuning instead of using a fixed feature extractor. Since later layers learn more task-specific features, we unfroze only the last 25% of the backbone to adapt details like beaks and wing bars and left the earlier layers fixed.

We used a dynamic unfreezing strategy that got the total number of parameters in our backbones (e.g., EVA-02, ViT) and released the gradients for the final 25% of the network. These layers were trained with a differential learning rate ($\eta_{backbone} = 1e^{-5}$ vs. $\eta_{head} = 1e^{-3}$) to avoid catastrophic forgetting (where fine-tuning overwrites useful features learned during pretraining).

This experiment also did not lead to any improvements, most likely because of the size of the training data. The dataset was simply too small to refine the deep features without overfitting, so the pretrained weights still performed better than the fine-tuned ones.

3.5.6. ViT Improvement: CV-Ensembling and Crow-Focused Fine-Tuning

Guided by our class-wise error analysis, we ran a targeted benchmark to address systematic confusions that were hidden by aggregate metrics. In particular, per-species accuracies revealed that *American Crow* and *Fish Crow* were among the weakest classes and were frequently confused with each other. To maximize supervision while keeping a robust evaluation protocol, we merged the provided training and validation splits and performed stratified 5-fold cross-validation on this combined labeled set, using the fold models both for model selection and as a natural ensemble at inference.

As a starting point, we reused our previous best `train_neural` configuration: a frozen ViT used purely as an embedding extractor, followed by a simple linear classification head. This baseline is fast to train and already strong overall, but the fold-by-fold, per-class breakdown confirmed that the crow pair remained a consistent failure mode, suggesting that the frozen embedding space did not separate these two visually similar species reliably enough.

We then introduced a more complex recipe (implemented in the Github branch `neural_ex_unfrozen`) aimed specifically at improving fine-grained separation: (i) using the 5 CV fold models as an ensemble by averaging their predictive distributions, (ii) applying a lightweight, class-dependent reweighting at inference to boost crow-related hypotheses when predictions are close, and (iii) partially unfreezing the ViT backbone and fine-tuning it with discriminative learning rates (smaller for the backbone, larger for the head) to adapt features without destabilizing

training. This comes at a clear cost in complexity, training time, and compute (multiple checkpoints, backpropagation through the backbone), so we explicitly compared it against the frozen-embedding baseline.

Empirically, the new setup improved crow separation noticeably, and this is reflected in the per-class 5-fold CV mean accuracies: *American Crow* increased from 68.3% to 86.7% and *Fish Crow* from 58.3% to 75.0%. However, on the held-out test set the overall accuracy remained essentially the same as with the simpler model, so the added complexity and training cost were not justified as a general replacement. Nevertheless, because it was substantially better for crow disambiguation, we later reused this stronger configuration in the pseudo-labeling stage.

3.5.7. Pseudo-Labeling

Per-class cross-validation on our best frozen EVA-02 backbone showed that most species were already saturated, but a small subset remained challenging. In particular, `Fish_Crow` achieved only 33.3% average accuracy, and `American_Crow` had low precision (64.7%), indicating frequent confusion and false positives despite strong overall performance (mean 5-fold CV accuracy 0.9376 ± 0.0049). Since collecting new labels is not allowed, we explored pseudo-labeling to increase training samples for the hardest classes.

We sourced additional unlabeled images from the NABirds dataset[3] and curated a set of $N = 1882$ images that contained all the bird species that we are called to distinguish. Those images were also cropped as explained previously.

To generate pseudo-labels, we used our crow-improved ViT configuration (partially unfrozen backbone), which performed better on the crow pair than the EVA-02 setup. We ran inference on the curated NABirds subset using a 5-fold CV ensemble (with horizontal-flip test-time augmentation) and stored, for each image, the predicted label $\hat{y}$ and confidence $\max_k p_\theta(y = k \mid x)$. We then applied a standard threshold rule with $\tau = 0.8$:

$$\tilde{y} = \hat{y} \quad \text{if} \quad \max_k p_\theta(y = k \mid x) \geq \tau.$$

We restricted pseudo-labels to the seven non-saturated classes identified by the EVA-02 error analysis (`American_Crow`, `Fish_Crow`, `Brewer_Blackbird`, `Rusty_Blackbird`, `Lazuli_Bunting`, `Black_billed_Cuckoo`, `Yellow_billed_Cuckoo`) and materialized them as class folders under `data/pseudo_labels/`. This produced 439 pseudo-labeled images in total: 57 `American_Crow`, 21 `Fish_Crow`, 47 `Brewer_Blackbird`, 30 `Rusty_Blackbird`, 84 `Lazuli_Bunting`, 39 `Black_billed_Cuckoo`, and 61 `Yellow_billed_Cuckoo`.

We retrained the EVA-02 frozen-backbone model with the pseudo-labeled images added to the training set, using the same 5-fold CV protocol. The mean CV accuracy stayed at 0.9376, but the key target class `Fish_Crow` decreased from 0.333 to 0.316 mean accuracy, while `American_Crow` remained around 0.93.

The lack of improvement is consistent with (i) residual label noise under domain shift (our labeled data vs. NABirds), where even high-confidence predictions can be wrong for the hardest fine-grained confusions, and (ii) imbalance in the added pseudo-labels (e.g., nearly twice as many `American_Crow` as `Fish_Crow`), which can bias the decision boundary. Given the unchanged overall accuracy and degraded Fish Crow performance, we did not proceed with this approach. However, the pseudo-labeled samples were included in the training of the crow separator.

3.5.8. Binary Crow Separator

The main classifier occasionally confuses *American Crow* and *Fish Crow*. Since these two classes are visually similar, we trained a dedicated **binary crow separator** and used it as a post-processing step to rerank only the test images predicted as one of the two crow classes. The implementation details can be found in the Github branch `crow-separator`.



Figure 5. Similarity of American and Fish crows

Training data (real + pseudo labels). The separator was fine-tuned on the curated crow training images and augmented with pseudo-labeled crow images. Pseudo-labeled samples were down-weighted using a reduced `pseudo_profile_weight = 0.5` (i.e., each pseudo example contributes half the weight of a real example)

Models. We trained two independent binary crow-separator models using the same EVA02-Large backbone at 448×448, but with different heads/objectives and model-selection criteria: (i) a fine-tuned backbone with a linear classification head trained with focal loss, strong data augmentation, and horizontal-flip TTA, selecting the best

[3] https://dl.allaboutbirds.org/nabirds

checkpoint by validation *balanced accuracy*; and (ii) a fine-tuned backbone with an ArcFace (margin-based) head trained with light augmentation (and label smoothing), using horizontal-flip TTA, selecting by validation *balanced accuracy*. For the final ensemble, we averaged the two models' predicted probabilities and applied the separator only to test samples initially predicted as one of the two crow classes; all non-crow predictions were left unchanged.

Ensembling and reranking. At inference time, we applied the separator *only* to samples whose predicted class was one of the two crow classes. For each such image, each separator model produced a probability vector over the two classes, and we formed an ensemble by averaging probabilities:

$$p_{\text{ens}}(y \mid x) \;=\; \frac{1}{2}\sum_{m=1}^{2} p_m(y \mid x), \qquad \hat{y} = \arg\max_{y} p_{\text{ens}}(y \mid x).$$

The final predicted crow label $\hat{y}$ replaced the original crow label in the submission, while all non-crow predictions were left unchanged. This yielded our final crow-disambiguation strategy.

3.6. Selected Model vs. Actual Best

3.6.1. Selected Model: ArcFace EVA-02 Large

During the competition, our strongest single model appeared to be the EVA-02 Large Vision Transformer trained with an ArcFace Head at 448×448 resolution. Based on the Public Leaderboard score of 0.93, this was the model we selected for final evaluation.

Increasing the input size to 448×448 (up from the standard 224 or 384) allowed the model to resolve finer details. Moreover, unlike standard cross-entropy, ArcFace not only separates classes but also encourages features from the same class to be closer together and features from different classes to be farther apart. Training for 50 epochs allowed the model to converge on these harder margins. However, the final Private Leaderboard revealed that this model had overfitted to the public subset, dropping to a score of ≈ 0.855.

3.6.2. Best Model: ViT-384 (Linear)

Contrary to our expectations, our simpler baseline, the standard ViT-384 with a simple Linear Head was much better at generalization. It scored lower on the Public Leaderboard (0.90), but kept that exact score of **0.90** on the Private Test set.

Computational Efficiency. Beyond accuracy, the ViT-384 (Linear Head) is also very efficient computationally. The ViT-384 model is quite lightweight: on a simple NVIDIA RTX 3050 6GB Laptop GPU, training for 30 epochs with a batch size of 32 took around 10 minutes. This also proves that fine-grained classification does not require large-scale compute resources; for this dataset, a frozen backbone with a well-tuned linear was both very efficient in training and produced the best results.

Final score	Description
90%	Vision Transformer (ViT-384) with Standard Linear Head (Simple Baseline).
89%	ViT-384 Improvement: CV-Ensembling and Crow- Focused Fine-Tuning
89%	EVA-02 Large (448px) with Linear Head (No ArcFace) and Extended Training
87%	EVA-02 Large (448px) with arcface and extra pseudo-labelled training data
87%	EVA-02 Large (448px) with Linear Head (No ArcFace)
87%	EVA-02 Large (448px) with arcface, extended Training and crow-separator reranking
77%	EfficientNet-B0 & Logisitic Regression

Table 2. Submissions sorted by highest final score.

4. Conclusions

This work studied fine-grained bird species recognition under a strict low-data constraint (20 classes, $\sim$1k labeled training images) and evaluated a broad spectrum of approaches, from hand-crafted descriptors (HOG, Fisher Vectors) to modern pretrained backbones.

Beyond aggregate metrics, qualitative inspection with the FiftyOne App proved essential to understand *why* models fail. Filtering incorrect and low-confidence samples revealed that errors concentrate in a small set of visually near-identical species pairs (e.g., American vs. Fish Crow) and are often exacerbated by occlusion, pose variation, and residual background context. This analysis motivated targeted attempts such as a dedicated crow separator and pseudo-labeling, and it also helped us distinguish confusions from isolated label-noise or outlier cases.

Across experiments, the most consistent improvements came from increasing signal-to-noise rather than increasing head complexity: cropping to reduce background reliance, preserving aspect ratio via padding (SquarePad), and using higher input resolutions (384–448) to retain fine-grained cues like feather micro-textures and beak morphology. In contrast, several seemingly well-motivated additions did not translate into better generalization (MLP heads on frozen features, broad mixtures of experts, naive soft-voting under calibration mismatch, and pseudo-labeling under domain shift), suggesting that performance is primarily limited by data coverage and the intrinsic ambiguity of certain class

pairs.

Our final evaluation on the private test set showed an important generalization issue in fine-grained classification. While the ArcFace EVA-02 model achieved 93.0% accuracy on the public validation set, its performance dropped to 85.5% on the private test set.

In contrast, a simpler baseline like the Vision Transformer (ViT-384) with a standard linear classification head was much more stable, achieving 90.0% accuracy on both the public and private sets.

These results have taught us a good lesson. In low-data regimes like in our case, trying to aggressively push class separation through losses such as ArcFace can reduce robustness. The "simpler" model, with less aggressive constraints, learned a more robust, generalized representation of the species. In the end, the complexity was not just unnecessary; it was detrimental.

Finally, our targeted experiments indicate a productive direction for future work: replacing "global" complexity with *localized* improvements guided by error analysis (e.g., specialized rerankers for the hardest confusions, better probability calibration, and more conservative semi-supervised learning that explicitly controls label noise and class imbalance). More broadly, the results emphasize that in fine-grained tasks, careful preprocessing and robust evaluation can matter as much as model choice, and that validation gains should be interpreted cautiously when the test distribution is not guaranteed to match the labeled splits. That is the case especially here, where the limited size of the labeled dataset (training and validation) makes the problem intrinsically harder and increases sensitivity to overfitting and distribution shift.

References

[1] Navneet Dalal and Bill Triggs. Histogram of oriented gradients (hog) in computer vision. *Towards Data Science*, 2019. Online article. 6

[2] Jiankang Deng, Jia Guo, Jing Yang, Niannan Xue, Irene Kotsia, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):5962–5979, 2022. 4

[3] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021. 4

[4] Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. Eva-02: A visual representation for neon genesis. *Image and Vision Computing*, 149:105171, 2024. 4

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015. 3

[6] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s, 2022. 3

[7] Tal Ridnik, Emanuel Ben-Baruch, Asaf Noy, and Lihi Zelnik-Manor. Imagenet-21k pretraining for the masses, 2021. 3

[8] Jorge Sánchez, Florent Perronnin, Thomas Mensink, and Jakob Verbeek. Image classification with the fisher vector: Theory and practice. *International journal of computer vision*, 105(3):222–245, 2013. 6

[9] scikit-image team. Fisher vector feature encoding. scikit-image documentation (stable), Example gallery, 2025. Accessed: 2026-02-02. 6

[10] Mingxing Tan and Quoc V. Le. Efficientnet: Rethinking model scaling for convolutional neural networks. *CoRR*, abs/1905.11946, 2019. 3

[11] Ross Wightman. Pytorch image models. `https://github.com/rwightman/pytorch-image-models`, 2019. 3

[12] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. Convnext v2: Co-designing and scaling convnets with masked autoencoders, 2023. 3

[13] Weihao Yu, Chenyang Si, Pan Zhou, Mi Luo, Yichen Zhou, Jiashi Feng, Shuicheng Yan, and Xinchao Wang. Metaformer baselines for vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(2):896–912, 2024. 3